什么时候值得让多个 Agent 协作?

从单体执行到工作流与 AUTO，
我们测的是：质量收益能否覆盖
额外的 token 和时间。



Claude Code
单主会话（允许分发 subagent）



Gold Band
工作流 / AUTO

判断依据

质量

是否通过独立测试
与整体验收。

成本

token 与
端到端耗时。

适用边界

从局部问题
到长程工程。

执行框架对照，不是模型能力排名。

结论：中短任务用 Direct，长任务用编排

中短题：编排质量略好或持平，但 token 与耗时更高；长题更能体现分工与整体验收的价值。

轻量 局部修复	DeepSWE	修复真实仓库中的独立问题	参考补丁 443–1,031 行	+
轻量 独立功能	zstd-decoder	实现一个完整的解码器	参考实现 2,373 行	+
中量 模块重建	mailauth	组合实现多种邮件认证能力	参考实现 18,881 行	+
重量 跨模块工程	Kubernetes	用 Rust 重写并完成整体验收	参考实现 133,220 行	+
超长 系统级重建	Ruff	重建代码检查器，持续长程执行	参考实现 约 25 万 行	+

长任务的收益

质量更好，token 差距可反转。

AUTO 在 Kubernetes 完整通过验收，token 约少 47%，但仍更慢。

点击左侧任务，查看具体题面与验收内容。

题目越长、越需要分工和整体验收，工作流与 AUTO 的价值越明显。

参考建议：预计新增/修改超过 5 万行、或上下文压缩 3 次及以上的任务，优先工作流 / AUTO。

短任务的收益，还不够覆盖开销

AUTO / Claude Code · 输出 token

2.4×

多通过 3 题；
输出开销约为单体的 2.4 倍。

同题对照	Claude Code	Gold Band AUTO
通过题数	12 / 29	15 / 29
输出 token	4.5M	10.7M
平均耗时 / 题	22 分钟	37 分钟

zstd-decoder：工作流同分

SWE-Marathon / 预定义轻量工作流
质量持平，token 与等待仍更多。

执行方式	通过测试	记录 token	Agent 耗时
Claude Code	43 / 43	约 64M	37 分钟
轻量工作流	43 / 43	约 126M	63 分钟

质量略好或持平，token 与耗时更高。

短任务，Direct 优先。

2.4× 仅指输出 token，不是总量。29 题配对、每臂每题一次，p≈0.5；不作普遍提升结论。来源：阶段汇报 README。

mailauth：质量接近，AUTO 开销更高

与 Claude Code 相比，两轮 AUTO 均未稳定拉开质量差距，token 与 Eval 耗时却都更高。

第一轮

2026.10.07 完成

验收与开销	Claude Code	AUTO
隐藏测试	684 / 702	694 / 702
总 token	336.6M	407.9M
Eval 耗时	2h 42m	3h 55m

AUTO 多通过 10 项；
token 约 1.21×，耗时约 1.45×。

第二轮

2026.10.08 完成

验收与开销	Claude Code	AUTO
隐藏测试	702 / 702	701 / 702
总 token	397.6M	699.5M
Eval 耗时	2h 30m	8h 09m

两边几乎满分；
token 约 1.76×，耗时约 3.26×。

协作开销，尚未换来稳定的质量收益。

中短任务，Direct 优先。

全部运行可见测试均为 851/851。总 token 含缓存读取，不等于金额；Eval 耗时取整至分钟。来源：阶段汇报 README。

长任务中，完整验收开始体现编排价值

整体验收 · Gold Band AUTO

1,407

项通过 / 共 1,407 项

完整通过

同题对照	Claude Code	Gold Band AUTO
通过测试	1,405 / 1,407	1,407 / 1,407
记录 token	约 325.8M	约 173M
Agent 耗时	97 分钟	165 分钟

AUTO 完整通过，记录 token 约少 47%。

耗时仍更长。

加入整体验收规则后的单次同题复测；token 不是账单金额。

Claude Code 未完成就主动退出；AUTO 无需额外闸门，持续推进

Claude Code · 无闸门首段

主动退出

hidden 通过率仅 8.2%（5 / 61）
仍能继续提升，却结束了任务。

约 309M 总 token　　8h 02m Eval 耗时

AUTO 经故障恢复后持续推进；这里比较执行方式，最终验收尚未完成。

AUTO · 编排持续运行

无需额外添加
防止提前退出的提交闸门。

程序接续任务、分工与验收，
不依赖主会话持续自我催促。

▶ 查看首段记录与测试口径

长程执行由编排持续接力，不靠额外闸门阻止提前退出。

来源：汇报 README · Ruff；首段已封存，后续 running 为 2026.10.13 快照，非实时结果。

goal 解决续跑，不解决上下文容量

单体应用中持续使用 goal，会不断触碰到主编排会话的上下文窗口限制，导致漂移。

goal 约束「何时算完成」；长程执行还需要把任务状态、交接证据和验收边界放到会话之外管理。

风险如何形成

主会话持续累积

需求、历史与决策不断叠加。

触顶后压缩历史

goal 不会扩大上下文窗口。

待办与验收约束漂移

约束可能在压缩与续跑中弱化。

继续执行的条件，与长期保留约束的能力，需要分别解决。

模型提出拆分，程序持续编排

模型　理解任务，提出 single / fanout 与后继路由

Runtime　校验运行图 · 维护状态 · 调度

Agent A	Agent B	Agent C
局部上下文	局部上下文	局部上下文

分工执行 ↓ 合并产物

验收 Agent · 整体检查　未通过 → 返工路由

职责示意；采用 single 还是 fanout，由模型根据任务提出。

把上下文分担出去

各 Agent 处理局部任务。程序维护状态与调度，不靠一个主模型会话持续承载全部执行历史。

合并后仍需整体验收

验收 Agent 对照原始目标检查最终产物；发现遗漏时，交给后继任务修复并复验。

模型参与计划与判断；Runtime 承接运行状态与持续调度。

来源：汇报 README · 架构分析；对应 AI-DYNAMIC 设计与 Workflow DSL。示意图不代表性能测量。

按任务形态，选择执行模式

执行模式	适用任务	选择重点
Direct	中短任务，目标集中 单 Agent 能覆盖主要工作	减少额外协调， 优先控制 token 与时间。
工作流	步骤可以预定义 重复流程与固定质量关口	把稳定步骤固化， 在明确关口检查质量。
AUTO	目标明确，路径动态 跨模块长任务，需要拆分与整体验收	按任务动态分工， 合并后检查整体交付。

题目越长、越需要分工和整体验收，**工作流与 AUTO 的优势越明显**：
质量更完整，token 差距缩小甚至占优。

把协作成本，花在单个 Agent 难以完整覆盖的任务上。

参考建议：预计新增/修改超过 5 万行、或上下文压缩 3 次及以上的任务，优先工作流 / AUTO；其余优先 Direct。